CS 271P · Artificial Intelligence · Fall 2026

PA 1,2B: Movement Range

Submission Instructions

You will implement one search function and submit it to Gradescope, where it is auto-graded. This handout is the complete specification: read it, fill in the provided stub, test locally, and submit.

At a Glance

- **Submit on Gradescope:** exactly one file, `reachable_targets.py` (this exact name) — the file itself, not a folder, archive, or notebook.
- **Implement** one function, `reachable_targets(grid, A, B, base_movement, path)`, using the **Python 3 standard library only** (no `numpy`, etc.).
- **Graded on correctness first, then relative speed** — and you will be asked a short **written question about your own code** (see *How Your Work Is Graded*).

THE PROBLEM

A square-grid board game has two characters left, **A** and **B**, and it is **A's turn**. An action limits A to **half** of its movement this turn (rounded up), and the turn must **end on a tile adjacent to B** — that is how A reaches B. Movement is **8-directional** (king moves: orthogonal *or* diagonal), each step costing **1**. **B's tile and every wall (obstacle) are blocked** — they cannot be entered or crossed.

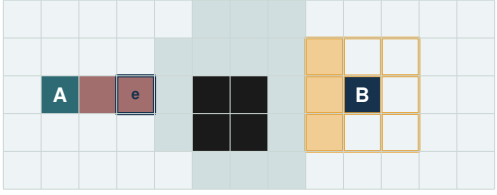
First, A is forced along a fixed **path** (a sequence of tiles, *excluding* A's own starting tile). After walking it, A has some movement **budget** left, measured from the path's end. From that end, your job is to report the two things the player needs:

- `adjacent_reachable` — every tile **next to B** that A could move to *directly* within the remaining budget. *These are the tiles where A could actually finish its turn.*
- `next_waypoints` — every tile A could step to next and *still* finish next to B within the budget. *These are the tiles that keep the goal reachable — a "where could I move?" shortlist.*

Tiles are written `(x, y)` with `x` **= column** and `y` **= row**, where **row 0 is at the top**.

A WORKED EXAMPLE

Take this board. **A** starts at `(1,2)` and is forced along the crimson path `(2,2)` → `(3,2)`, so the path end is `e` = `(3,2)`. Say `base_movement` = `14`. Then the half-budget is $M = \lceil 14/2 \rceil = 7$; the path used $T = 2$ moves; so the **remaining budget is** $R = M - T = 5$, measured from `e`.



crimson = forced path teal = `next_waypoints` amber = `adjacent_reachable` = B's ring dark = walls.

Reading it off (remember $R = 5$):

- `adjacent_reachable` $= \{(8,1),(8,2),(8,3)\}$ — only B's *near* side. Each is next to B *and* reachable from `e` in ≤ 5 moves (going around the walls). B's far side is out of budget: e.g. `(10,2)` is 7 moves from `e`.
- `next_waypoints`: a tile w qualifies when **(moves** `e`$\to w$**)** $+$ **(moves** $w \to$ **the nearest tile next to B)** $\leq R$.
  - `(5,1)` **qualifies**: 2 moves from `e`, then 3 more on to the B-adjacent `(8,1)` — $2 + 3 = 5 \leq R$. Stepping here keeps the goal reachable.
  - `(4,4)` **does not**: also 2 moves from `e`, but the nearest tile next to B is then 4 away — $2 + 4 = 6 > R$, so committing here would strand A.

  (`e` itself and the reachable ring tiles count as waypoints too.)

</> YOUR FUNCTION

Implement this function (the provided `reachable_targets.py` contains exactly this stub):

```
def reachable_targets(grid, A, B, base_movement, path):
    """Return (next_waypoints, adjacent_reachable) as two sets of (x, y) tiles."""
    ...
    return next_waypoints, adjacent_reachable
```

The rules, precisely

- Movement is 8-directional (king moves), cost 1 each. **Blocked** tiles are every tile in `grid.obstacles` *and B's own tile*; they cannot be entered or crossed.
- $M = \lceil$`base_movement`$/2 \rceil$ (half the movement, rounded up). The path *excludes* A, so it costs $T =$ `len(path)` moves; the **remaining budget** is $R = M - T$, measured from the path end $e =$ `path[-1]` (or $e = A$ if `path` is empty). If $R < 0$, both sets are empty.
- **Adjacent to B** means a **passable king-neighbor of B** — A stands *next to* B, not on it (B's tile is blocked). Call these tiles the *ring*.
- `adjacent_reachable`**:** every *ring* tile whose shortest distance from e is $\leq R$.
- `next_waypoints`**:** every passable tile w with $\mathrm{dist}(e, w) + \mathrm{dist}(w,$ nearest ring tile$) \leq R$ — that is, A can reach w *and* from w still reach a ring tile, in $\leq R$ moves total. (This set includes e and the reachable ring tiles themselves.)

> **Solution hint.** Start off by tracing routes outward from `e`, following each as far as the remaining budget allows, to discover which tiles stay within reach.

Provided to your function (do *not* import or redefine these — they are supplied when your code runs):

- `grid.in_bounds(t)` — is tile `t = (x, y)` on the board?
- `grid.king_neighbors(t)` — the up-to-8 in-bounds king-move neighbors of `t`. **It does *not* skip walls or B**, so filter blocked tiles out yourself.
- `grid.W`, `grid.H` — board width (columns) and height (rows).
- `grid.obstacles` — the set of wall tiles.
- `cheb(p, q)` — king (Chebyshev) distance between two tiles, *ignoring walls*.

Return value: two sets of `(x, y)` tuples — `next_waypoints` first, then `adjacent_reachable`. Empty sets are valid answers (e.g. when B can't be reached in time).

RESTRICTIONS

Important: Submissions that break these rules may fail to import and score **0**.

- The file must be named `reachable_targets.py` and define a function named `reachable_targets` with **exactly** the signature above.
- **Python 3 standard library only** — no `numpy`, no `matplotlib`, no third-party packages. (`math`, `collections`, etc. are fine.)
- Do **not** import or redefine the provided names (`grid`, `cheb`); use the `grid` object and `cheb` exactly as given.
- Return **two sets** of `(x, y)` tuples, in the stated order.
- Do **not** read files, access the network, print, or rely on global state.
- Be reasonably efficient. Your solution must stay under a **256 MB memory limit** — generous for this task; only pathologically wasteful solutions exceed it (if yours does, you get a message saying so).

TEST BEFORE YOU SUBMIT

The package includes `test_local.py`. From the assignment folder, run:

```
python3 test_local.py
```

It checks your function for **(1) correctness** — on fixed edge-case boards plus many random boards, showing a picture of any board you get wrong — and **(2) speed** against a deliberately *naive* baseline (the slow, brute-force reference that simply checks every tile on the board one at a time). You may also edit the boards in `test_local.py` to try your own.

A longer test, like the final grading run. Add the option `-final`:

```
python3 test_local.py --final
```

It checks 500 random boards, where the plain test checks 40, and it times your solution on 40 boards of about 160 by 160 tiles. The boards are **new on every run**. They are not the boards of the final grading run. The large boards are only timed, because the naive baseline is too slow to check answers on them. See *How Your Work Is Graded* for the final grading run.

Important: Passing the local test means you are **correct** and **faster than the naive baseline** — on Gradescope that is at least the ∼**70% range**. It does **not** guarantee full marks: the higher speed tiers (and the small top bonus) are decided on Gradescope against reference solutions not shown here.

Grading environment. Your code runs on Gradescope in a **Linux (Ubuntu) container** under **Python 3**. Because you use only the **Python 3 standard library**, it will grade correctly as long as it runs under **Python 3** (not Python 2) with **no third-party packages** — the same way `test_local.py` runs it here. If it passes locally under Python 3, it will run for grading.

HOW YOUR WORK IS GRADED

Your submission is graded on **correctness** and then **relative speed**, on **randomly generated boards**, so a solution must work in general and not only on fixed cases.

The assignment is scored out of **100 points**. Each percentage below is also a number of points, so the 70% floor is 70 points and the 105% bonus is 105 points.

- **Memory limit.** Your solution must stay under **256 MB** (generous for this task); a solution that exceeds it is reported to you and earns no speed credit.
- **Correctness first.** For each board you can see the board and your answer versus the expected answer. An **incorrect** solution scores **0**; a **correct** one earns at least the **70% floor**.
- **Then speed scales your score up.** The *naive* baseline (the slow brute-force one in your local tester) is the floor: a correct solution **no faster than naive** stays at **70%**; the faster your solution, the higher its tier, up to **100%**.
- **The 105% bonus is a top-tier threshold — not "fastest wins".** A small **105%** bonus sits above 100%, awarded **only to solutions fast enough to clear the top speed tier**. Simply being the fastest submission does **not** by itself earn it — you have to reach that threshold.
- **A buffer works in your favor.** Speed is judged with a **buffer** that absorbs ordinary run-to-run timing noise and reasonable coding-style differences, so you won't lose a tier to luck. (It isn't secret — ask if you'd like to know how it works.)

> **Important: Be ready to explain your code.** You will get a short **written question** (quiz-style, in lecture) asking you to explain part of your implementation — how it works, or an analysis of it (for example, why it is correct, or why it is efficient). Make sure you understand what you wrote and why.

Two grading runs. Your work is graded twice.

1. **While the assignment is open**, every submission is graded at once, on freshly generated random boards. Submit as often as you like. Use `test_local.py` to iterate quickly.
2. **After the deadline**, once every submission is in, your active submission (your last one, unless you choose another on Gradescope) is graded again by a **final run**. It uses the same kinds of boards, but many more of them, and every student gets the same boards. A solution that reaches **95%** or higher is also timed on much larger boards, about 160 by 160 tiles, and its speed tier there replaces the earlier one. It can go up or down. **Your grade is the score from the final run.**

SUBMITTING ON GRADESCOPE

1. Open the `PA 1,2B: Movement Range` assignment on **Gradescope**.
2. Upload your `reachable_targets.py` (the file only).
3. Wait for the autograder to finish, then read the correctness results.
4. Resubmit as often as you like before the deadline.

These instructions are subject to change. Any updates will be announced in class and posted on Canvas.